Gen AI - Unit3 Submission1-Handson Assignment1

Name	Diya D Bhat
SRN	PES2UG23CS183
Section	C
Date	31/03/26

1) Advanced RAG & Retrieval Enhancement Techniques

Part 1a: Why Naïve RAG Fails

```
# Our knowledge base — 5 documents
corpus = [
    "Transformers use self-attention mechanisms to process sequences in parallel.",          # doc_0
    "BERT is a bidirectional encoder trained using masked language modelling.",              # doc_1
    "The BM25 algorithm ranks documents based on term frequency and inverse document frequency.",  # doc_2
    "Gradient descent is an optimization technique used to minimize the loss function.",     # doc_3
    "Neural networks learn by adjusting weights through backpropagation.",                   # doc_4
]

# Our query: deliberately uses a synonym that dense models handle OK, but BM25 struggles with
query_vocab_mismatch = "How do neural nets learn?"

# Our query: deliberately uses an exact keyword that BM25 excels at
query_exact_match = "BM25 term frequency"

print(f"Corpus size: {len(corpus)} documents")
print(f"Query 1 (semantic): '{query_vocab_mismatch}'")
print(f"Query 2 (keyword):  '{query_exact_match}'")
```

```
Corpus size: 5 documents
Query 1 (semantic): 'How do neural nets learn?'
Query 2 (keyword):  'BM25 term frequency'
```

```
# --- Naïve Dense-Only Retrieval ---
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

corpus_embeddings = model.encode(corpus, convert_to_numpy=True)

def dense_retrieve(query, top_k=3):
    q_vec = model.encode([query], convert_to_numpy=True)[0]
    scores = corpus_embeddings @ q_vec / (
        np.linalg.norm(corpus_embeddings, axis=1) * np.linalg.norm(q_vec)
    )
    ranked = np.argsort(scores)[::-1][:top_k]
    return [(i, scores[i], corpus[i]) for i in ranked]

print("=== Dense Retrieval Results ===")
print(f"\nQuery: '{query_vocab_mismatch}'")
for rank, (idx, score, doc) in enumerate(dense_retrieve(query_vocab_mismatch), 1):
    print(f"  Rank {rank} [doc_{idx}] score={score:.4f}: {doc[:70]}")

print(f"\nQuery: '{query_exact_match}'")
for rank, (idx, score, doc) in enumerate(dense_retrieve(query_exact_match), 1):
    print(f"  Rank {rank} [doc_{idx}] score={score:.4f}: {doc[:70]}")
```

```
WARNING:tensorflow:From C:\Users\diyab\AppData\Local\Programs\Python\Python313\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_
softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

=== Dense Retrieval Results ===

Query: 'How do neural nets learn?'
  Rank 1 [doc_4] score=0.7550: Neural networks learn by adjusting weights through backpropagation.
  Rank 2 [doc_3] score=0.3656: Gradient descent is an optimization technique used to minimize the los
  Rank 3 [doc_1] score=0.2918: BERT is a bidirectional encoder trained using masked language modellin

Query: 'BM25 term frequency'
  Rank 1 [doc_2] score=0.5714: The BM25 algorithm ranks documents based on term frequency and inverse
  Rank 2 [doc_0] score=0.1448: Transformers use self-attention mechanisms to process sequences in par
  Rank 3 [doc_4] score=0.1129: Neural networks learn by adjusting weights through backpropagation.
```

Part 1b: Sparse Retrieval — BM25

```
from rank_bm25 import BM25Okapi

# BM25 works on tokenized (word-split) documents
tokenized_corpus = [doc.lower().split() for doc in corpus]

bm25 = BM25Okapi(tokenized_corpus)

def bm25_retrieve(query, top_k=3):
    tokenized_query = query.lower().split()
    scores = bm25.get_scores(tokenized_query)
    ranked = np.argsort(scores)[::-1][:top_k]
    return [(i, scores[i], corpus[i]) for i in ranked]

print("=== BM25 Retrieval Results ===")
print(f"\nQuery: '{query_vocab_mismatch}'")
for rank, (idx, score, doc) in enumerate(bm25_retrieve(query_vocab_mismatch), 1):
    print(f"  Rank {rank} [doc_{idx}] score={score:.4f}: {doc[:70]}")

print(f"\nQuery: '{query_exact_match}'")
for rank, (idx, score, doc) in enumerate(bm25_retrieve(query_exact_match), 1):
    print(f"  Rank {rank} [doc_{idx}] score={score:.4f}: {doc[:70]}")
```

```
=== BM25 Retrieval Results ===

Query: 'How do neural nets learn?'
  Rank 1 [doc_4] score=1.2259: Neural networks learn by adjusting weights through backpropagation.
  Rank 2 [doc_3] score=0.0000: Gradient descent is an optimization technique used to minimize the los
  Rank 3 [doc_2] score=0.0000: The BM25 algorithm ranks documents based on term frequency and inverse

Query: 'BM25 term frequency'
  Rank 1 [doc_2] score=2.9625: The BM25 algorithm ranks documents based on term frequency and inverse
  Rank 2 [doc_4] score=0.0000: Neural networks learn by adjusting weights through backpropagation.
  Rank 3 [doc_3] score=0.0000: Gradient descent is an optimization technique used to minimize the los
```

Part 1c: Dense Retrieval — SBERT and ColBERT

```
    ColBERT MaxSim scoring.
    Each query token finds its best matching document token.
    Score = sum of those max similarities.
    """
    query_tokens = query.lower().split()       # tokenize (simplified: word-level)
    doc_tokens   = document.lower().split()

    # Encode each token as a vector
    q_vecs = encoder.encode(query_tokens, convert_to_numpy=True)   # shape: (|Q|, dim)
    d_vecs = encoder.encode(doc_tokens,   convert_to_numpy=True)   # shape: (|D|, dim)

    # Normalize
    q_vecs = q_vecs / np.linalg.norm(q_vecs, axis=1, keepdims=True)
    d_vecs = d_vecs / np.linalg.norm(d_vecs, axis=1, keepdims=True)

    # Similarity matrix: (|Q|, |D|)
    sim_matrix = q_vecs @ d_vecs.T

    # MaxSim: for each query token, take max across all doc tokens
    max_sims = sim_matrix.max(axis=1)    # shape: (|Q|,)

    return float(max_sims.sum())

print("=== ColBERT-style Late Interaction Scores ===")
query = "How do neural nets learn?"
print(f"Query: '{query}'\n")
scores = []
for i, doc in enumerate(corpus):
    s = colbert_score(query, doc)
    scores.append(s)
    print(f"  doc_{i} score={s:.4f}: {doc[:70]}")

best = np.argmax(scores)
print(f"\nTop document: doc_{best} — '{corpus[best]}'")
```

```
=== ColBERT-style Late Interaction Scores ===
Query: 'How do neural nets learn?'

  doc_0 score=1.8954: Transformers use self-attention mechanisms to process sequences in par
  doc_1 score=2.1215: BERT is a bidirectional encoder trained using masked language modellin
  doc_2 score=1.8315: The BM25 algorithm ranks documents based on term frequency and inverse
  doc_3 score=1.9923: Gradient descent is an optimization technique used to minimize the los
  doc_4 score=3.3560: Neural networks learn by adjusting weights through backpropagation.

Top document: doc_4 — 'Neural networks learn by adjusting weights through backpropagation.'
```

Part 1d: Hybrid Retrieval — BM25 + SBERT with Reciprocal Rank Fusion

```python
# ============================================================
# NUMERICAL EXAMPLE: Full BM25 + SBERT + RRF walkthrough
# Query: "How do neural nets learn?"
# ============================================================

import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer

corpus = [
    "Transformers use self-attention mechanisms to process sequences in parallel.",
    "BERT is a bidirectional encoder trained using masked language modelling.",
    "The BM25 algorithm ranks documents based on term frequency and inverse document frequency.",
    "Gradient descent is an optimization technique used to minimize the loss function.",
    "Neural networks learn by adjusting weights through backpropagation.",
]

query = "How do neural nets learn?"

# ---- STEP 1: BM25 Scores ----
tokenized_corpus = [doc.lower().split() for doc in corpus]
bm25 = BM25Okapi(tokenized_corpus)
bm25_scores = bm25.get_scores(query.lower().split())

print("STEP 1: BM25 Raw Scores")
print("-" * 60)
for i, s in enumerate(bm25_scores):
    print(f"  doc_{i}: {s:.4f}  |  {corpus[i][:60]}")

bm25_ranked = np.argsort(bm25_scores)[::-1]
bm25_ranks  = {doc_id: rank+1 for rank, doc_id in enumerate(bm25_ranked)}

print("\nBM25 Ranking (best → worst):", list(bm25_ranked))
```

```
STEP 1: BM25 Raw Scores
------------------------------------------------------------
  doc_0: 0.0000  |  Transformers use self-attention mechanisms to process sequen
  doc_1: 0.0000  |  BERT is a bidirectional encoder trained using masked languag
  doc_2: 0.0000  |  The BM25 algorithm ranks documents based on term frequency a
  doc_3: 0.0000  |  Gradient descent is an optimization technique used to minimi
  doc_4: 1.2259  |  Neural networks learn by adjusting weights through backpropa

BM25 Ranking (best → worst): [np.int64(4), np.int64(3), np.int64(2), np.int64(1), np.int64(0)]
```

```python
# ---- STEP 2: SBERT Dense Scores ----
sbert = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

doc_vecs   = sbert.encode(corpus, convert_to_numpy=True)
query_vec  = sbert.encode([query], convert_to_numpy=True)[0]

doc_vecs_norm   = doc_vecs   / np.linalg.norm(doc_vecs,   axis=1, keepdims=True)
query_vec_norm  = query_vec  / np.linalg.norm(query_vec)

sbert_scores = doc_vecs_norm @ query_vec_norm

print("STEP 2: SBERT Cosine Scores")
print("-" * 60)
for i, s in enumerate(sbert_scores):
    print(f"  doc_{i}: {s:.4f}  |  {corpus[i][:60]}")

sbert_ranked = np.argsort(sbert_scores)[::-1]
sbert_ranks  = {doc_id: rank+1 for rank, doc_id in enumerate(sbert_ranked)}

print("\nSBERT Ranking (best → worst):", list(sbert_ranked))
```

```
STEP 2: SBERT Cosine Scores
------------------------------------------------------------
  doc_0: 0.2637  |  Transformers use self-attention mechanisms to process sequen
  doc_1: 0.2918  |  BERT is a bidirectional encoder trained using masked languag
  doc_2: 0.0809  |  The BM25 algorithm ranks documents based on term frequency a
  doc_3: 0.3656  |  Gradient descent is an optimization technique used to minimi
  doc_4: 0.7550  |  Neural networks learn by adjusting weights through backpropa

SBERT Ranking (best → worst): [np.int64(4), np.int64(3), np.int64(1), np.int64(0), np.int64(2)]
```

```
# ---- STEP 3: Reciprocal Rank Fusion ----

K = 60  # RRF smoothing constant

print("STEP 3: Reciprocal Rank Fusion (k=60)")
print("-" * 80)
print(f"{'Doc':<8} {'BM25 rank':<12} {'SBERT rank':<13} {'RRF(BM25)':<14} {'RRF(SBERT)':<14} {'Total RRF'}")
print("-" * 80)

rrf_scores = {}
for doc_id in range(len(corpus)):
    rb = bm25_ranks[doc_id]
    rs = sbert_ranks[doc_id]
    rrf_bm25  = 1.0 / (K + rb)
    rrf_sbert = 1.0 / (K + rs)
    total     = rrf_bm25 + rrf_sbert
    rrf_scores[doc_id] = total
    print(f"  doc_{doc_id}  {rb:<12} {rs:<13} {rrf_bm25:<14.6f} {rrf_sbert:<14.6f} {total:.6f}")

# Final ranking
final_ranked = sorted(rrf_scores, key=rrf_scores.get, reverse=True)

print("\n" + "=" * 80)
print("FINAL HYBRID RANKING")
print("=" * 80)
for rank, doc_id in enumerate(final_ranked, 1):
    print(f"  #{rank}  doc_{doc_id} (RRF={rrf_scores[doc_id]:.6f}): {corpus[doc_id]}")
```

```
STEP 3: Reciprocal Rank Fusion (k=60)
--------------------------------------------------------------------------------
Doc      BM25 rank    SBERT rank    RRF(BM25)      RRF(SBERT)     Total RRF
--------------------------------------------------------------------------------
  doc_0  5            4             0.015385       0.015625       0.031010
  doc_1  4            3             0.015625       0.015873       0.031498
  doc_2  3            5             0.015873       0.015385       0.031258
  doc_3  2            2             0.016129       0.016129       0.032258
  doc_4  1            1             0.016393       0.016393       0.032787

================================================================================
FINAL HYBRID RANKING
================================================================================
  #1  doc_4 (RRF=0.032787): Neural networks learn by adjusting weights through backpropagation.
  #2  doc_3 (RRF=0.032258): Gradient descent is an optimization technique used to minimize the loss function.
  #3  doc_1 (RRF=0.031498): BERT is a bidirectional encoder trained using masked language modelling.
  #4  doc_2 (RRF=0.031258): The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #5  doc_0 (RRF=0.031010): Transformers use self-attention mechanisms to process sequences in parallel.
```

```
# ---- Comparison Table: BM25 vs SBERT vs Hybrid ----

print("COMPARISON: BM25 vs SBERT vs Hybrid (for query: 'How do neural nets learn?')")
print("=" * 90)
print(f"{'Rank':<6} {'BM25 top doc':<50} {'SBERT top doc':<50}")
print("-" * 90)
for i in range(len(corpus)):
    bm25_doc = corpus[bm25_ranked[i]][:45]
    sbert_doc = corpus[sbert_ranked[i]][:45]
    print(f"  #{i+1}   {bm25_doc:<50} {sbert_doc:<50}")

print()
print("Hybrid Final Ranking:")
for rank, doc_id in enumerate(final_ranked, 1):
    print(f"  #{rank}  {corpus[doc_id]}")
```

```
COMPARISON: BM25 vs SBERT vs Hybrid (for query: 'How do neural nets learn?')
==========================================================================================
Rank   BM25 top doc                                       SBERT top doc
------------------------------------------------------------------------------------------
  #1   Neural networks learn by adjusting weights th      Neural networks learn by adjusting weights th
  #2   Gradient descent is an optimization technique      Gradient descent is an optimization technique
  #3   The BM25 algorithm ranks documents based on t      BERT is a bidirectional encoder trained using
  #4   BERT is a bidirectional encoder trained using      Transformers use self-attention mechanisms to
  #5   Transformers use self-attention mechanisms to      The BM25 algorithm ranks documents based on t

Hybrid Final Ranking:
  #1  Neural networks learn by adjusting weights through backpropagation.
  #2  Gradient descent is an optimization technique used to minimize the loss function.
  #3  BERT is a bidirectional encoder trained using masked language modelling.
  #4  The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #5  Transformers use self-attention mechanisms to process sequences in parallel.
```

```python
# Test with a keyword-heavy query where BM25 should dominate

query2 = "BM25 term frequency ranking"

bm25_scores2  = bm25.get_scores(query2.lower().split())
bm25_ranked2  = np.argsort(bm25_scores2)[::-1]
bm25_ranks2   = {doc_id: rank+1 for rank, doc_id in enumerate(bm25_ranked2)}

query2_vec    = sbert.encode([query2], convert_to_numpy=True)[0]
query2_norm   = query2_vec / np.linalg.norm(query2_vec)
sbert_scores2 = doc_vecs_norm @ query2_norm
sbert_ranked2 = np.argsort(sbert_scores2)[::-1]
sbert_ranks2  = {doc_id: rank+1 for rank, doc_id in enumerate(sbert_ranked2)}

rrf_scores2 = {}
for doc_id in range(len(corpus)):
    rrf_scores2[doc_id] = 1/(K + bm25_ranks2[doc_id]) + 1/(K + sbert_ranks2[doc_id])
final_ranked2 = sorted(rrf_scores2, key=rrf_scores2.get, reverse=True)

print(f"Query: '{query2}'\n")
print(f"{'Doc':<8} {'BM25 rank':<12} {'SBERT rank':<13} {'RRF Score':<12} Document")
print("-" * 90)
for doc_id in range(len(corpus)):
    print(f"  doc_{doc_id}  {bm25_ranks2[doc_id]:<12} {sbert_ranks2[doc_id]:<13} {rrf_scores2[doc_id]:.6f}    ")

print("\nHybrid Final Ranking:")
for rank, doc_id in enumerate(final_ranked2, 1):
    print(f"  #{rank}  doc_{doc_id}: {corpus[doc_id]}")
```

```
Query: 'BM25 term frequency ranking'

Doc      BM25 rank    SBERT rank    RRF Score    Document
------------------------------------------------------------------------------------------
  doc_0  5            5             0.030769   Transformers use self-attention mechanisms to process sequen
  doc_1  4            3             0.031498   BERT is a bidirectional encoder trained using masked languag
  doc_2  1            1             0.032787   The BM25 algorithm ranks documents based on term frequency a
  doc_3  3            4             0.031498   Gradient descent is an optimization technique used to minimi
  doc_4  2            2             0.032258   Neural networks learn by adjusting weights through backpropa

Hybrid Final Ranking:
  #1  doc_2: The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #2  doc_4: Neural networks learn by adjusting weights through backpropagation.
  #3  doc_1: BERT is a bidirectional encoder trained using masked language modelling.
  #4  doc_3: Gradient descent is an optimization technique used to minimize the loss function.
  #5  doc_0: Transformers use self-attention mechanisms to process sequences in parallel.
```

Part 1e: Building a Full Hybrid Retriever (Reusable Function)

```
Query: 'How do neural nets learn?'
  → doc_4 (RRF=0.03279): Neural networks learn by adjusting weights through backpropagation.
  → doc_3 (RRF=0.03226): Gradient descent is an optimization technique used to minimize the loss function.

Query: 'BM25 term frequency ranking'
  → doc_2 (RRF=0.03279): The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  → doc_4 (RRF=0.03226): Neural networks learn by adjusting weights through backpropagation.

Query: 'attention in transformers'
  → doc_0 (RRF=0.03279): Transformers use self-attention mechanisms to process sequences in parallel.
  → doc_4 (RRF=0.03226): Neural networks learn by adjusting weights through backpropagation.
```

2) Re-Ranking & Query Expansion

```python
import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer, CrossEncoder

# Shared corpus from Notebook 1 — extended for richer demo
corpus = [
    "Transformers use self-attention mechanisms to process sequences in parallel.",
    "BERT is a bidirectional encoder trained using masked language modelling.",
    "The BM25 algorithm ranks documents based on term frequency and inverse document frequency.",
    "Gradient descent is an optimization technique used to minimize the loss function.",
    "Neural networks learn by adjusting weights through backpropagation.",
    "Retrieval Augmented Generation combines a retriever with a language model to produce grounded answers.",
    "Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.",
    "Quantization reduces model size by representing weights in lower bit formats such as INT8.",
    "The attention mechanism computes a weighted sum of value vectors based on query-key similarity.",
    "Large language models are trained on massive text corpora to learn general-purpose representations.",
]

print(f"Corpus: {len(corpus)} documents loaded.")
```

```
Corpus: 10 documents loaded.
```

Part 2a: Re-Ranking with Cross-Encoders

```
Models loaded.
  Bi-encoder:    all-MiniLM-L6-v2 (first-stage retrieval)
  Cross-encoder: ms-marco-MiniLM-L-6-v2 (re-ranking)
```

```
STAGE 1 — Bi-Encoder Top-5 Candidates
----------------------------------------------------------------------
  #1  [score=0.5273] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2  [score=0.4970] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3  [score=0.3708] BERT is a bidirectional encoder trained using masked language modelling.
  #4  [score=0.3478] Transformers use self-attention mechanisms to process sequences in parallel.
  #5  [score=0.3418] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

```
STAGE 2 — Cross-Encoder Re-Ranking
----------------------------------------------------------------------
  #1  [CE score=3.3522] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2  [CE score=-4.4683] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3  [CE score=-5.4407] Transformers use self-attention mechanisms to process sequences in parallel.
  #4  [CE score=-6.9237] BERT is a bidirectional encoder trained using masked language modelling.
  #5  [CE score=-10.4939] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.

--- Before vs After Re-Ranking ---
Bi-Encoder Rank 1:             The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
Cross-Encoder Rank 1:          The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
```

Part 2b: Query Expansion

```
Original Query:
  'How does attention work in language models?'

Hypothetical Document (generated by Gemini):
  In language models, attention is a mechanism that allows the model to selectively focus on specific parts of the input sequence when generating output.
This is achieved through the use of attention weights, which are calculated based on the similarity between the input sequence and the model's internal s
tate. The attention weights are then used to compute a weighted sum of the input sequence, effectively "paying attention" to the most relevant parts of t
he input. This allows the model to capture long-range dependencies and relationships in the input data, improving its ability to generate coherent and co
ntextually relevant output.
```

```
Query: 'How does attention work in language models?'

Raw Query Retrieval (Top 3):
  #1 [score=0.5273] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2 [score=0.4970] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3 [score=0.3708] BERT is a bidirectional encoder trained using masked language modelling.

HyDE Retrieval (Top 3):
  #1 [score=0.6168] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2 [score=0.5145] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3 [score=0.4330] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

```
Generated query variants:
  (original): How does attention work in language models?
  (variant 1): What is the mechanism behind attention in language models that enables them to focus on specific parts of the input?
  (variant 2): How do language models utilize attention to selectively weigh the importance of different input elements during processing?
  (variant 3): What is the role of attention in language models in terms of dynamically allocating computational resources to relevant input components?

Multi-Query retrieved 5 unique documents (union of all variants):
  [doc_8] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  [doc_9] Large language models are trained on massive text corpora to learn general-purpose representations.
  [doc_1] BERT is a bidirectional encoder trained using masked language modelling.
  [doc_0] Transformers use self-attention mechanisms to process sequences in parallel.
  [doc_4] Neural networks learn by adjusting weights through backpropagation.
```

Part 2c: Full Production Pipeline — Query Expansion → Hybrid → Re-Rank → Generate

```python
# --- Step functions ---

def expand_query(query: str) -> str:
    """Use HyDE to expand the query."""
    return hyde_chain.invoke({"query": query})

def hybrid_retrieve(expanded_query: str) -> list[str]:
    """Retrieve top-5 using Hybrid (BM25+SBERT+RRF)."""
    return hybrid.retrieve(expanded_query, top_k=5)

def rerank(data: dict) -> list[str]:
    """Re-rank retrieved docs using cross-encoder."""
    query = data["original_query"]
    docs  = data["candidates"]
    pairs  = [[query, doc] for doc in docs]
    scores = cross_encoder.predict(pairs)
    ranked = np.argsort(scores)[::-1][:3]  # Keep top-3
    return [docs[i] for i in ranked]

def format_context(docs: list[str]) -> str:
    return "\n\n".join(f"[Document {i+1}]\n{doc}" for i, doc in enumerate(docs))

print("Pipeline components defined.")
```

```
Pipeline components defined.
```

```
Query: 'How does attention work in language models?'
======================================================================
[1] HyDE Expansion: In language models, attention is a mechanism that allows the model to selectively focus on specific parts of the input sequence when
generating output...

[2] Hybrid Retrieval (5 candidates):
     #1: The attention mechanism computes a weighted sum of value vectors based on query-
     #2: Large language models are trained on massive text corpora to learn general-purpo
     #3: Transformers use self-attention mechanisms to process sequences in parallel.
     #4: The BM25 algorithm ranks documents based on term frequency and inverse document
     #5: Fine-tuning adapts a pre-trained model to a specific downstream task using task-

[3] After Re-Ranking (top 3):
     #1: The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
     #2: Large language models are trained on massive text corpora to learn general-purpose representations.
     #3: Transformers use self-attention mechanisms to process sequences in parallel.

[4] Final Answer:
     According to Document 1, the attention mechanism computes a weighted sum of value vectors based on query-key similarity.
'According to Document 1, the attention mechanism computes a weighted sum of value vectors based on query-key similarity.'
```

```
Query: 'What is the purpose of backpropagation?'
======================================================================
[1] HyDE Expansion: Backpropagation is a fundamental algorithm used in the training of artificial neural networks, particularly in supervised learning. I
ts primary purpos...

[2] Hybrid Retrieval (5 candidates):
     #1: Gradient descent is an optimization technique used to minimize the loss function
     #2: Neural networks learn by adjusting weights through backpropagation.
     #3: The attention mechanism computes a weighted sum of value vectors based on query-
     #4: The BM25 algorithm ranks documents based on term frequency and inverse document
     #5: BERT is a bidirectional encoder trained using masked language modelling.

[3] After Re-Ranking (top 3):
     #1: Neural networks learn by adjusting weights through backpropagation.
     #2: Gradient descent is an optimization technique used to minimize the loss function.
     #3: The attention mechanism computes a weighted sum of value vectors based on query-key similarity.

[4] Final Answer:
     Neural networks learn by adjusting weights through backpropagation.
'Neural networks learn by adjusting weights through backpropagation.'
```

3) Generation Enhancement Techniques

```
%pip install python-dotenv transformers peft datasets langchain langchain-groq langchain-google-genai accelerate
# bitsandbytes is for quantization on GPU; on CPU we simulate the math instead
# %pip install bitsandbytes --quiet  # uncomment if on a GPU machine
```

```
Requirement already satisfied: python-dotenv in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (1.2.1)
Requirement already satisfied: transformers in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (4.57.6)
Collecting peft
  Downloading peft-0.18.1-py3-none-any.whl.metadata (14 kB)
Collecting datasets
  Downloading datasets-4.8.4-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: langchain in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (1.2.10)
Requirement already satisfied: langchain-groq in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (1.1.2)
Requirement already satisfied: langchain-google-genai in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (4.2.0)
Collecting accelerate
  Downloading accelerate-1.13.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: filelock in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (3.18.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformer
s) (0.36.2)
Requirement already satisfied: numpy>=1.17 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (2.3.3)
Requirement already satisfied: packaging>=20.0 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (2026.1.
15)
Requirement already satisfied: requests in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (2.32.5)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformer
s) (0.22.2)
Requirement already satisfied: safetensors>=0.4.3 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from huggingface-hub<1.0,>=
0.34.0->transformers) (2025.5.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from huggingface-h
ub<1.0,>=0.34.0->transformers) (4.15.0)
Requirement already satisfied: psutil in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from peft) (6.1.1)
Requirement already satisfied: torch>=1.13.0 in c:\users\diyab\appdata\local\programs\python\python313\lib\site-packages (from peft) (2.7.1)
Collecting pyarrow>=21.0.0 (from datasets)
```

```
Google API Key not found — enter it now:  ·····
Groq API Key not found — enter it now:  ·····
GOOGLE key loaded: yes
GROQ   key loaded: yes
```

Part 3a: Fine-Tuning

```
Original weight W:     64x64 = 4,096 parameters
LoRA matrices A+B:     4x64 + 64x4 = 512 parameters
LoRA overhead:         12.50% of original

Original output:  [ 0.0417  -0.05841  0.07145  0.00424]
LoRA output:      [ 0.0417  -0.05841  0.07145  0.00424]
(outputs identical at init because B=0, so ΔW = B@A = 0)
```

```
Loading a tiny model for PEFT demo (distilgpt2)...
trainable params: 147,456 || all params: 82,060,032 || trainable%: 0.1797
C:\Users\diyab\AppData\Local\Programs\Python\Python313\Lib\site-packages\pef
the target module is `Conv1D`. Setting fan_in_fan_out to True.
  warnings.warn(
```

Part 3b: Data Resolution & Precision

```
Original value:  1.3456789
FP32:            1.3456789255  (error: 2.55e-08)
FP16:            1.3457031250  (error: 2.42e-05)
BF16:            1.3437500000  (error: 1.93e-03)

--- Memory footprint of a 1-billion parameter model ---
  FP32        : 3.73 GB
  FP16/BF16   : 1.86 GB
  INT8        : 0.93 GB
  INT4        : 0.47 GB
```

```
FP16 overflow example:
  Value:   70000.0
  FP32:    70000.0  (correct)
  FP16:    inf  ← OVERFLOW! Becomes inf
  BF16:    70144.0  (safe — wide exponent range)
```

Part 3c: Quantization

```
Linear Quantization Demo (FP32 → INT8 → FP32)
================================================================
Scale factor S = 0.007111
Zero point   Z = -95

Index    FP32         INT8       Reconstructed    Error
----------------------------------------------------------------
  [0]     0.496714     -25         0.497787   0.001073
  [1]    -0.138264    -114        -0.135114   0.003151
  [2]     0.647689      -4         0.647123   0.000566
  [3]     1.523030     119         1.521805   0.001225
  [4]    -0.234153    -128        -0.234671   0.000518
  [5]    -0.234137    -128        -0.234671   0.000534
  [6]     1.579213     127         1.578695   0.000518
  [7]     0.767435      13         0.768014   0.000579

Max quantization error: 0.003151
Mean quantization error: 0.001020

Memory: FP32=32 bytes → INT8=8 bytes (4.0x reduction)
```

```
Model Size Comparison for Common LLMs
================================================================
Model                Params       FP32 (GB)    FP16 (GB)    INT8 (GB)    INT4 (GB)
----------------------------------------------------------------
distilgpt2           0.07B        0.25         0.12         0.06         0.03
GPT-2                0.12B        0.44         0.22         0.11         0.05
BERT-Base            0.11B        0.41         0.20         0.10         0.05
Llama-3.2-1B         1.00B        3.73         1.86         0.93         0.47
Llama-3.1-8B         8.00B        29.80        14.90        7.45         3.73
Llama-3.1-70B        70.00B       260.77       130.39       65.19        32.60
```

```
Quantization Trade-off (100 weights)
Bits     MSE (error)        Size (bytes)    Compression
------------------------------------------------------------
  32       0.23258863           400.0             1.0x
  16       0.00000000           200.0             2.0x
  8        0.00003263           100.0             4.0x
  4        0.00870766           50.0              8.0x
  2        0.19430648           25.0              16.0x
C:\Users\diyab\AppData\Local\Temp\ipykernel_20052\792760889.p
  q_vals = np.clip(np.round(W / S + Z), q_min, q_max).astype(
```

Part 3d: Mixture of Experts (MoE)

```
Defined 5 experts: ['math', 'code', 'creative', 'science', 'general']
```

```
Router Test:
------------------------------------------------------------
  🔢 [math      ] What is the derivative of x² + 3x?
  💻 [code      ] Write a Python function to reverse a linked list.
  ✍️ [creative  ] Write a short poem about autumn leaves.
  🔬 [science   ] How does photosynthesis work?
  🌐 [general   ] What time is it in Tokyo?
```

```
🔢 Expert: MATH
Query:  What is the integral of sin(x)dx?
Answer: To find the integral of $\sin(x)$, we can use the following formula:

$$\int \sin(x) \, dx = -\cos(x) + C$$

where $C$ is the constant of integration.

To derive this formula, we can use the following steps:

1. Recall the definition of the derivative of the cosine function:

$$\frac{d}{dx} \cos(x) = -\sin(x)$$

2. Integrate both sides of the equation with respect to $x$:

$$\int \frac{d}{dx} \cos...
======================================================================
💻 Expert: CODE
Query:  Write a recursive function to compute Fibonacci numbers in Python.
Answer: **Fibonacci Recursive Function in Python**
```python
def fibonacci(n: int) -> int:
    """
    Compute the nth Fibonacci number using recursion.

    Args:
    n (int): The position of the Fibonacci number to compute.

    Returns:
    int: The nth Fibonacci number.
    """
    # Base cases: F(0) = 0, F(1) = 1
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # Recursive case: ...
======================================================================
💻 Expert: CODE
Query:  How do transformer models use attention mechanisms?
Answer: **Transformer Model with Attention Mechanism**
============================================================

The Transformer model, introduced in the paper "Attention Is All You Need" by Vaswani et al. in 2017, revolutionized the field of natural language proce
ssing (NLP) by eliminating the need for recurrent neural networks (RNNs) and convolutional neural networks (CNNs). The Transformer model uses self...
======================================================================
```

```
Query: 'Explain recursion.'
======================================================================

💻 CODE Expert:
**Recursion in Programming**
==============================

Recursion is a fundamental concept in programming where a function calls itself repeatedly until it reaches a base case that stops the recursion. This t
echnique allows us to break down complex problems into simpler ones and solve them in...
----------------------------------------------------------------------

✍️ CREATIVE Expert:
In the Labyrinthine Hall of Mirrors

Imagine a grand, ornate hallway with a seemingly endless series of reflections staring back at you. Each mirror perfectly duplicates the image of the ha
llway, creating a dizzying maze of identical corridors that extend infinitely in every direction. You wander de...
----------------------------------------------------------------------

🔬 SCIENCE Expert:
Recursion is a fundamental concept in computer science and mathematics that can be a bit tricky to grasp at first, but I'll try to explain it in a way t
hat's easy to understand.

**What is Recursion?**

Recursion is a problem-solving technique where a function or method calls itself repeatedly until...
----------------------------------------------------------------------
```
```